

Git Workflow Guidelines

Standard rules for 42 projects

Summary: Authoritative Git workflow rules used across 42 projects. Project subjects may specify which sections apply.

Version: 2.0

Contents

I Tips: Rebase and Corrections **2**

II Git Workflow Guidelines **3**

II.1 Commit Messages 3

II.1.1 One Commit per Feature 3

II.2 Branching Strategy 4

II.2.1 General rules 4

II.2.2 Merge Requests must reflect the real development process 4

II.2.3 Flexibility 4

II.3 Default Branch 4

II.4 Merge Strategy 4

II.5 Integrating Main Changes 5

Chapter I

Tips: Rebase and Corrections

We strongly recommend using interactive rebase to fix previous commits if mistakes are made on development branches:

```
git rebase --i HEAD~n
```

This allows you to:

- Edit commit messages
- Squash commits
- Reorder commits
- Fix content in previous commits

Use rebase to maintain a clean and logical commit history before merging to main.

Chapter II

Git Workflow Guidelines

These rules define the expected Git workflow for 42 projects. They ensure clarity, reproducibility, and a clean history aligned with 42's evaluation standards.



Unless a project subject explicitly narrows the scope, all sections in this document apply. The treenity project requires every section to be respected.

II.1 Commit Messages

All commits must follow the Conventional Commits specification:
https://www.conventionalcommits.org/
A commit should represent one feature, one fix, or one scoped change only.

```
feat(broker): add topic creation
fix(consumer): handle connection timeouts
chore: update build configuration
```

II.1.1 One Commit per Feature

A single commit must contain:

- one feature, fix, or logical change,
- no unrelated modifications.



Students who do not respect these Git workflow guidelines will receive a grade of 0. You can check your commit format and branch naming by running gitinette on your session (provided as an attachment with the subject). During evaluation, the evaluator runs gitinette -evaluation, which additionally samples random commits for a manual message/diff coherence check.

II.2 Branching Strategy

II.2.1 General rules

- Every new feature, bugfix, or significant change must be developed in its own branch.
- Branch names should match the commit type when possible:

```
feat/producer-client
fix/message-persistence
chore/refactor-protocol
```

II.2.2 Merge Requests must reflect the real development process

- A merge request must show how the feature was actually built.
- Intermediate commits are acceptable and encouraged for traceability.
- Squashing commits is not allowed.

II.2.3 Flexibility

- Minor fixes or adjustments inside a feature branch are acceptable.
- Cross-feature or unrelated changes must never be mixed. If necessary, create a dedicated branch.

II.3 Default Branch

The `main` branch must remain stable.
Only the following changes are allowed directly on `main`:

- small non-breaking fixes (typos, documentation, trivial corrections),
- corrections that do not alter feature logic.

Everything else must go through a feature branch and a merge request.

II.4 Merge Strategy

All branches must be integrated into `main` using a merge commit (no fast-forward). This preserves the development graph and makes feature boundaries explicit.

```
git merge --no-ff <branch>
```

II.5 Integrating Main Changes

When you need to integrate changes from `main` into your feature branch, use rebase or fast-forward operations only. Merge commits should only occur when integrating feature branches back into `main`.

```
git rebase main
git rebase --onto main <old-base> <feature-branch>
git pull --rebase origin main
```

Using rebase or equivalent approaches:

- Maintains a linear commit history on feature branches
- Keeps merge commits exclusively for feature integration
- Makes the development timeline clearer and easier to follow
- Simplifies conflict resolution by dealing with conflicts commit by commit

Do not use merge commits to bring main changes into feature branches, as this creates unnecessary complexity in the git history.